INT234: PREDICTIVE ANALYTICS PROJECT REPORT

(Project Semester: January – October 2026)

AIRLINE PASSENGER SATISFACTION PREDICTION USING MACHINE LEARNING

Predicting passenger satisfaction through exploratory data analysis and machine learning classification models

Submitted by

Thupakula Hemanth Yadav

Registration No. 12409760

Programme and Section – B.Tech CSE (K3P24MW)

Course Code: INT234

Under the Guidance of

Dr. Madhu Bala (UID: 31770)



Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

October 2026

DECLARATION

I, Thupakula Hemanth Yadav student of B.Tech Computer Science and Engineering (Data Science) under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 31**-10-2026**

Registration No**: 12409760**

Signature:hemanthyadav

CERTIFICATE

This is to certify that Thupakula Hemanth Yadav bearing Registration No. **12409760** has completed **INT234** project titled, "Airline Passenger Satisfaction Prediction Using Machine Learning" under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature: T.hemanth

Name of the Supervisor: Dr. Madhu Bala

U.Id: **31770**

Designation of the Supervisor:

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab.

Date: 31-10-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty supervisor, **Dr. Madhu Bala**, for the guidance, encouragement and constructive feedback that shaped this project from the selection of the topic to the preparation of this report.

I am thankful to the School of Computer Science and Engineering at Lovely Professional University, Phagwara, for providing the academic environment and learning resources needed to complete this Data Science minor project.

I also acknowledge the contributors of the Airline Passenger Satisfaction dataset made available on Kaggle, and the open-source communities behind Python, pandas, Matplotlib, seaborn and scikit-learn, whose tools made the analysis and model building possible.

Finally, I thank my family and friends for their continued support and motivation throughout the course of this work.

Name : Thupakula Hemanth Yadav

Registration No. **:** 12409760

TABLE OF CONTENTS

1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives of the Project 1
1.4 Scope and Limitations 2
1.5 Tools and Technologies 2
1.6 Organisation of the Report 3
2. Source of Dataset 4
2.1 Dataset Origin 4
2.2 Attributes in the Dataset 4
2.3 Size of the Dataset 5
2.4 Target Variable 6
3. EDA Process 7
3.1 Overview of the Workflow 7
3.2 Loading and Initial Inspection 7
3.3 Data Preprocessing 8
3.4 Statistical Exploration 9
3.5 Feature Engineering and Data Preparation 10
4. Analysis on Dataset 13
4.1 Overall Passenger Satisfaction Distribution 13
4.2 Passenger Age Distribution 14
4.3 Satisfaction Across Travel Classes 15
4.4 Satisfaction by Type of Travel 17
4.5 Satisfaction by Customer Type 18
4.6 Service Rating Analysis 19
4.7 Correlation Analysis of Selected Features 21
4.8 Arrival Delay Analysis by Passenger Satisfaction 22
4.9 Logistic Regression 24
4.10 Decision Tree Classifier 26
4.11 Random Forest Classifier 28
4.12 Support Vector Machine (SVM) 30
4.13 Model Evaluation and Comparison 32
5. Conclusion 35
5.1 Data Preparation 35
5.2 Exploratory Findings and Satisfaction Patterns 35

5.3 Machine Learning Implementation and Model Comparison 36
5.4 Key Learning Outcomes 36
6. Future Scope 37
6.1 Hyperparameter Tuning and Cross-Validation 37
6.2 Feature Selection and Model Optimisation 37
6.3 Interactive Dashboard 37
6.4 Web Application and Real-Time Prediction 38
7. References 39
Appendix A: Python Source Code 40

LIST OF FIGURES

Figure 1: Distribution of Passenger Satisfaction 14
Figure 2: Age Distribution of Passengers 15
Figure 3: Satisfaction by Travel Class 17
Figure 4: Satisfaction by Type of Travel 18
Figure 5: Satisfaction by Customer Type 19
Figure 6: Average Service Ratings by Satisfaction 21
Figure 7: Correlation Between Service Ratings and Flight Delays 22
Figure 8: Arrival Delay by Satisfaction 24
Figure 9: Logistic Regression - Confusion Matrix 26
Figure 10: Decision Tree (First Three Levels) 28
Figure 11: Accuracy Comparison of Classification Models 34

LIST OF TABLES

Table 1: Tools and libraries used in the project 2
Table 2: Dataset source details 4
Table 3: Attributes of the dataset used in the project 4
Table 4: Dataset size and data quality summary 5
Table 5: Workflow of the project script 7
Table 6: Outlier counts by the IQR rule 10
Table 7: Age groups created by pd.cut 10
Table 8: Inputs supplied to each classification model 12
Table 9: Confusion matrix of Logistic Regression 25
Table 10: Evaluation metrics of Logistic Regression on the test set 25
Table 11: Classification report of Logistic Regression 25
Table 12: Evaluation metrics of the Decision Tree on the test set 27
Table 13: Classification report of the Decision Tree 28
Table 14: Confusion matrix of the Decision Tree 28
Table 15: Evaluation metrics of the Random Forest on the test set 29
Table 16: Classification report of the Random Forest 30
Table 17: Confusion matrix of the Random Forest 30
Table 18: Evaluation metrics of the Support Vector Machine on the test set 31
Table 19: Classification report of the Support Vector Machine 32
Table 20: Confusion matrix of the Support Vector Machine 32
Table 21: Comparison of the four classification models on the test set 33
Table 22: Completed work and proposed improvements 37

1. Introduction

1.1 Background

Customer satisfaction is one of the most closely monitored indicators in the airline industry. Air travel is a service in which passengers form an opinion from a long chain of experiences: booking a ticket, checking in, boarding, seat comfort, cabin cleanliness, food, entertainment, wireless connectivity, baggage handling and punctuality. A passenger who is dissatisfied at one or more of these touchpoints may switch carriers, which directly affects revenue and brand reputation. Airlines therefore collect post-flight survey data to learn which aspects of the journey matter most to their passengers.

Survey data of this kind is large and multi-dimensional, and it is difficult to interpret by inspection alone. Data science offers two complementary approaches: exploratory data analysis (EDA), which summarises and visualises the data to reveal patterns, and machine learning, which learns a mapping from passenger and flight attributes to a satisfaction outcome so that the outcome can be predicted for unseen passengers.

1.2 Problem Statement

Given a record describing a passenger (demographic details, type and class of travel, flight distance, ratings for on-ground and in-flight services, and flight delays), the task is to determine whether the passenger is **satisfied** or **neutral or dissatisfied**. This is a supervised binary classification problem. In addition to prediction, the project seeks to understand which factors are associated with satisfaction.

1.3 Objectives of the Project

The objectives of this minor project are as follows:

- To clean and prepare the airline passenger satisfaction dataset for analysis by removing irrelevant columns, correcting text inconsistencies, removing duplicate records and handling missing values.
- To study the overall distribution of passenger satisfaction and the age distribution of passengers.
- To compare satisfaction across travel class, type of travel and customer type.

- To compare the average ratings of fourteen service features between satisfied and neutral or dissatisfied passengers.
- To examine the relationships among selected service ratings and flight delays using a correlation heatmap, and to study arrival delay against satisfaction.
- To engineer an age-group feature and encode categorical variables for machine learning.
- To implement four classification algorithms (Logistic Regression, Decision Tree, Random Forest and Support Vector Machine), evaluate them on a held-out test set and identify the best-performing model.

1.4 Scope and Limitations

The project is restricted to the Kaggle airline passenger satisfaction dataset and to the four algorithms listed above. The work is an offline, single-run analysis: models are trained on one stratified 80:20 train–test split with fixed random seeds, and no hyperparameter search or cross-validation has been performed. The Support Vector Machine is trained on a subset of the training data for practical run-time reasons (Section 3.5 and Section 4.12). The findings describe associations present in the dataset and should not be read as proof of causation.

1.5 Tools and Technologies

The project is implemented entirely in Python. Table 1 lists the tools and libraries used and the role of each in the implementation.

Table 1: Tools and libraries used in the project

Tool / Library	Role in the project
Python	General-purpose programming language used for the entire pipeline.
pandas	Loading the CSV file, inspecting data, cleaning, grouping, cross-tabulation, feature creation and one-hot encoding.
Matplotlib	Creating and displaying figures , including the bar chart of average service ratings and the decision-tree plot.
seaborn	Statistical plots built on Matplotlib: countplot, histplot, boxplot, heatmap and barplot.
scikit-learn	Train–test split, StandardScaler, LabelEncoder, the four classifiers, and evaluation metrics.

Tool / Library	Role in the project
Spyder	Development environments for writing and running the script.

NumPy is not imported directly in the project script; it is used internally by pandas and scikit-learn.

1.6 Organisation of the Report

Chapter 2 describes the source and structure of the dataset. Chapter 3 documents the EDA process: data inspection, preprocessing, statistical exploration and feature engineering. Chapter 4 presents each analysis in the five-part format prescribed by the university (Introduction, General Description, Specific Requirements, Functions and Formulas, Analysis Results, and Visualization); it covers eight exploratory analyses, four machine learning models and the model comparison. Chapter 5 gives the conclusion, Chapter 6 the future scope, and Chapter 7 the references. Appendix A reproduces the Python source code.

2. Source of Dataset

2.1 Dataset Origin

The dataset used in this project was obtained from Kaggle, a public platform for datasets and data science competitions. It is an airline passenger satisfaction survey dataset in which each record corresponds to one passenger's flight experience. The CSV file used by the script is named "Airline Passenger Satisfaction.csv".

Table 2: Dataset source details

Item	Details
Platform	Kaggle
Dataset title	Airline Passenger Satisfaction
Dataset URL	https://tinyurl.com/2r39xcwp
Date accessed	15-10-2026

2.2 Attributes in the Dataset

The script and its outputs show that the dataset contains passenger attributes, flight attributes, fourteen service-rating attributes and one target attribute. Table 3 summarises the attributes referred to in the project code. Two columns, "Unnamed: 0" and "id", are present in the raw file but are removed during preprocessing.

Table 3: Attributes of the dataset used in the project

Attribute	Type	Description
Unnamed: 0	Index	Row index column from the raw file; removed.
id	Identifier	Passenger/record identifier; removed.
Customer Type	Categorical	Loyal Customer or disloyal Customer.
Age	Numerical	Age of the passenger in years.
Type of Travel	Categorical	Business travel or Personal Travel.
Class	Categorical	Travel class: Business, Eco or Eco Plus.
Flight Distance	Numerical	Distance of the flight.

Attribute	Type	Description
Service ratings (14)	Numerical (ordinal)	Inflight wifi service; Departure/Arrival time convenient; Ease of Online booking; Gate location; Food and drink; Online boarding; Seat comfort; Inflight entertainment; On-board service; Leg room service; Baggage handling; Checkin service; Inflight service; Cleanliness.
Departure Delay in Minutes	Numerical	Departure delay of the flight in minutes.
Arrival Delay in Minutes	Numerical	Arrival delay of the flight in minutes.
satisfaction	Categorical (target)	neutral or dissatisfied, or satisfied.

2.3 Size of the Dataset

The exact shape of the raw file is reported by the statement df.info() in the script. The model-evaluation outputs of the project allow the number of records used for machine learning to be established: the root node of the decision tree (Figure 10) contains 83,123 training samples, and the confusion matrix of Figure 9 accounts for 10,642 + 1,134 + 1,414 + 7,591 = 20,781 test samples. Together these give 103,904 records after cleaning, consistent with an 80:20 split. Table 4 summarises the dataset size and data-quality counts.

Table 4: Dataset size and data quality summary

Item	Value
Rows and columns in the raw file	(103904, 25)
Shape after dropping “Unnamed: 0” and “id”	(103904, 23)
Duplicate rows found	0
Shape after removing duplicates	(103904, 23)
Columns with missing values and counts	310
Records used for modelling (training + test)	103,904 (83,123 + 20,781), derived from Figures 9 and 10
Records in the test set by class	neutral or dissatisfied: 11,776; satisfied: 9,005 (from Figure 9)

2.4 Target Variable

The target variable is **satisfaction**, which has two classes: “neutral or dissatisfied” and “satisfied”. Because “neutral” and “dissatisfied” are combined, the problem is a binary classification task. During preparation, scikit-learn’s LabelEncoder converts the labels to integers in alphabetical order, so “neutral or dissatisfied” becomes 0 and “satisfied” becomes 1. In the metrics and confusion matrices of this report, “satisfied” is therefore the positive class.

3. EDA Process

Exploratory data analysis is the stage in which the structure, quality and patterns of a dataset are examined before modelling . This chapter documents the EDA workflow as implemented in the project script: inspection of the raw data, preprocessing, statistical exploration and feature engineering. The graphical analyses are presented in Chapter 4.

3.1 Overview of the Workflow

The script is organised into numbered steps. Table 5 maps each stage of the workflow to the operations and the principal Python functions used.

Table 5: Workflow of the project script

Stage	Operations	Principal functions
1. Load and inspect	Read CSV; view first rows, data types and descriptive statistics	pd.read_csv, head, info, describe
2. Drop columns	Remove “Unnamed: 0” and “id”	drop(columns=…, errors="ignore")
3. Clean text, remove duplicates	Strip spaces in text columns; delete duplicate rows	str.strip, duplicated, drop_duplicates
4. Missing values	Mean for numerical, mode for categorical columns	isnull().sum, fillna, mean, mode
5. Statistics	Correlation, covariance, skewness, kurtosis, IQR outliers	corr, cov, skew, kurt, quantile
6. Visualization	Eight analyses (Figures 1 to 8)	seaborn and Matplotlib plots
7. Objective summaries	Counts, percentages, cross-tabulations, grouped means	value_counts, crosstab, groupby
8. Feature engineering	Age group; one-hot encoding; label encoding	pd.cut, pd.get_dummies, LabelEncoder
9. Data preparation	Stratified 80:20 split; standardisation	train_test_split, StandardScaler
10. Modelling and evaluation	Four classifiers; metrics; comparison	scikit-learn estimators and metrics

3.2 Loading and Initial Inspection

The dataset is loaded into a pandas DataFrame using pd.read_csv(). Three inspection calls follow. df.head() displays the first five records to confirm that the file was read correctly. df.info() lists every column with its data type and non-null count, which immediately reveals the presence of missing values and shows which columns are numerical and which are text. df.describe(include="all") produces summary statistics for all columns: count, mean, standard deviation, minimum, quartiles and maximum for numerical columns, and count, number of unique values, top value and its frequency for categorical columns.

```
df = pd.read_csv("<path>/Airline Passenger Satisfaction.csv")
print(df.head())
df.info()
print(df.describe(include="all"))
```

The outputs of these three calls are the basis of Table 4 and of the attribute list in Table 3.

3.3 Data Preprocessing

Raw survey data is rarely ready for analysis. The preprocessing steps below were applied in the order in which they appear in the script.

3.3.1 Removing Irrelevant Columns

The raw file contains an unnamed index column (“Unnamed: 0”) and an identifier column (“id”). Neither describes the passenger’s experience; they are artefacts of how the file was exported. If retained, a model could treat them as informative features and learn patterns that have no real meaning. They were therefore removed with df.drop(columns=["Unnamed: 0", "id"], inplace=True, errors="ignore"). The argument errors="ignore" prevents an error if either column is absent, which keeps the script re-runnable.

3.3.2 Cleaning Text Values

For every text (object) column, leading and trailing spaces were removed with str.strip(). Category labels that differ only by invisible whitespace would otherwise be counted as different categories, which distorts counts, group means and one-hot encoding.

```
for column in df.select_dtypes(include="object").columns:
```

```
    df[column] = df[column].str.strip()
```

3.3.3 Detecting and Removing Duplicate Records

Duplicate rows give some observations more weight than others and can leak between the training and test sets, which inflates evaluation scores. The script counts duplicates with df.duplicated().sum() and removes them with df.drop_duplicates(). Duplicates are removed after the identifier column has been dropped, so rows are compared on their real attributes only. The number of duplicates found and the shape before and after removal are recorded in Table 4.

3.3.4 Handling Missing Values

The script first prints the count of missing values in each column with df.isnull().sum(). Columns are then split by data type using select_dtypes. Each numerical column that contains missing values is filled with the column mean, and each categorical column that contains missing values is filled with the column mode (most frequent value):

$$\bar{x} = (1 / n)\, \Sigma\, x_i \qquad mode = most\ frequently\ occurring\ value$$

Mean imputation keeps the column average unchanged and retains all records, which is preferable to deleting rows when only a small fraction of values is missing. The mode is used for categorical columns because a mean is undefined for text categories. A limitation of mean imputation is that it reduces the variance of the filled column and can be distorted by extreme values; this is relevant for delay columns, which contain large outliers (Section 4.8). The missing-value counts before and after imputation are reported in Table 4.

```
for column in numerical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mean())
for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mode()[0])
```

3.3.5 Checking Data Types and Consistency

Data types were checked with df.info() and used programmatically: select_dtypes(include="number") identifies columns for mean imputation, correlation, skewness and kurtosis, while select_dtypes(include="object") identifies text columns for stripping and mode imputation. After imputation the script prints the missing-value counts again to confirm that no missing values remain.

3.4 Statistical Exploration

Before plotting, the script computes numerical summaries of the cleaned data.

- **Correlation matrix** (df.corr(numeric_only=True)): the Pearson correlation coefficient r between each pair of numerical columns, ranging from −1 to +1.
- **Covariance matrix** (df.cov(numeric_only=True)): the joint variability of each pair of numerical columns, which depends on the scale of the variables.
- **Skewness** (skew) and **kurtosis** (kurt): skewness measures the asymmetry of a distribution; a positive value indicates a long right tail. Kurtosis measures tail heaviness relative to a normal distribution.
- **Outlier counts by the IQR rule** for Age, Flight Distance, Departure Delay in Minutes and Arrival Delay in Minutes.

$$r = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \sqrt{[\,\Sigma(x_i - \bar{x})^2 \cdot \Sigma(y_i - \bar{y})^2\,]}$$

$$IQR = Q_3 - Q_1 \qquad Lower\ fence = Q_1 - 1.5 \times IQR \qquad Upper\ fence = Q_3 + 1.5 \times IQR$$

A value is counted as an outlier when it lies below the lower fence or above the upper fence. The script only counts outliers; it does not remove or cap them, so all outlying records are retained for modelling. Table 6 is reserved for the outlier counts printed by the script.

Table 6: Outlier counts by the IQR rule

Column	Number of outliers
Age	0
Flight Distance	2291
Departure Delay in Minutes	14529
Arrival Delay in Minutes	13954

The skewness and kurtosis values and the full correlation and covariance matrices printed by the script. A selected subset of the correlation matrix is visualised in Figure 7.

3.5 Feature Engineering and Data Preparation

Machine learning algorithms require numeric input and benefit from informative features. The following transformations were performed after the exploratory analyses.

3.5.1 Age Grouping

A new categorical feature, “Age Group”, was created from the numerical Age column using pd.cut() with the bin edges 0, 24, 39, 59 and 100:

```
df["Age Group"] = pd.cut(df["Age"], bins=[0, 24, 39, 59, 100],
                         labels=["18-24", "25-39", "40-59", "60+"])
```

Table 7: Age groups created by pd.cut

Label	Age range implied by the bin edges
18-24	1 to 24 years (bin (0, 24])
25-39	25 to 39 years (bin (24, 39])
40-59	40 to 59 years (bin (39, 59])
60+	60 to 100 years (bin (59, 100])

Table 7 shows the resulting groups. Grouping converts a continuous variable into life-stage bands that are easier to interpret and that allow a model to capture non-linear age effects as separate indicator columns. It should be noted that the histogram in Figure 2 shows passengers younger than 18, and because the first bin starts at 0, such passengers also fall in the group labelled “18-24”. The label therefore denotes “24 and below” in practice. The original Age column is retained alongside the new feature.

3.5.2 Encoding Categorical Variables

Scikit-learn estimators cannot process text. All remaining text predictors, together with the new Age Group feature, were converted into 0/1 indicator columns with pd.get_dummies(X, drop_first=True), a form of one-hot encoding. With drop_first=True, one category of each variable is omitted because it can be inferred when all remaining indicators are zero; this avoids the redundancy known as the dummy-variable trap, which is especially harmful for Logistic Regression. The indicator columns visible in the decision tree of Figure 10, namely “Type of Travel_Personal Travel”, “Customer Type_disloyal Customer” and “Class_Eco”, are produced by this step. The total number of features after encoding .

3.5.3 Encoding the Target

The target column was separated from the predictors (X = df.drop("satisfaction", axis=1); y = df["satisfaction"]) and converted to integers with LabelEncoder. The script prints the mapping, which is neutral or dissatisfied = 0 and satisfied = 1.

3.5.4 Train–Test Split

The data was divided into training and test sets with train_test_split(X, y, test_size=0.2, random_state=42, stratify=y). The training set (80%) is used to fit the models and the test set (20%) is used only for evaluation. Stratification preserves the class proportions in both subsets, which matters because the classes are not perfectly balanced (Section 4.1). The fixed random_state makes the split reproducible. The split produced 83,123 training and 20,781 test records (Section 2.3).

3.5.5 Feature Scaling

A StandardScaler was fitted on the training features and then applied to both training and test features. Each feature is transformed as:

$$z = (x - \mu) / \sigma$$

where μ and σ are the mean and standard deviation of the feature in the training set. The scaler is fitted on the training data only, so no information from the test set influences preprocessing. Scaled features are used for Logistic Regression and the Support Vector Machine, whose optimisation and distance-based margins are sensitive to feature scale. The Decision Tree and Random Forest split on thresholds of individual features and are unaffected by monotonic scaling, so they use the unscaled features. Table 8 summarises which inputs each model receives.

Table 8: Inputs supplied to each classification model

Model	Feature input	Training rows used
Logistic Regression	Scaled (X_train_scaled)	All training rows
Decision Tree	Unscaled (X_train)	All training rows
Random Forest	Unscaled (X_train)	All training rows
Support Vector Machine	Scaled (X_train_scaled)	First 15,000 training rows

4. Analysis on Dataset

This chapter presents each analysis of the project in the five-part structure prescribed by the university: (i) Introduction, (ii) General Description, (iii) Specific Requirements, Functions and Formulas, (iv) Analysis Results and (v) Visualization. Sections 4.1 to 4.8 cover the exploratory analyses, Sections 4.9 to 4.12 cover the four machine learning models, and Section 4.13 compares the models. Where a value is described as approximate, it has been read from the bars or boxes of the figure and the exact value is available in the script output.

4.1 Overall Passenger Satisfaction Distribution

i. Introduction

The first step in any classification study is to examine the target variable. This analysis determines how many passengers are satisfied and how many are neutral or dissatisfied. The result shows whether the classes are balanced, which influences the choice of train–test strategy and evaluation metrics.

ii. General Description

The variable analysed is **satisfaction**, a categorical variable with two levels. A count plot shows the number of observations in each level as bars; the bar height equals the frequency of the category. Class imbalance occurs when one class is substantially larger than the other.

iii. Specific Requirements, Functions and Formulas

The chart was drawn with sns.countplot(data=df, x="satisfaction", palette=["skyblue", "orange"]) and formatted with plt.figure, plt.title, plt.xlabel, plt.ylabel, plt.xticks(rotation=10) and plt.tight_layout. Exact counts and percentages are produced by value_counts() and value_counts(normalize=True). The percentage of a class is:

$$\textit{Percentage of class} = (\textit{number of passengers in class} / \textit{total passengers}) \times 100$$

iv. Analysis Results

Figure 1 shows that “neutral or dissatisfied” is the larger class, at roughly 59,000 passengers, while “satisfied” contains roughly 45,000 passengers. The classes are therefore moderately imbalanced rather than severely so. The test-set composition of

11,776 and 9,005 passengers (Figure 9) corresponds to about 56.7% neutral or dissatisfied and 43.3% satisfied, and because the split was stratified, the full dataset has nearly the same proportions. Exact counts and percentages from the script.

Key observations: (a) More passengers are neutral or dissatisfied than satisfied. (b) The imbalance is modest, so accuracy remains a meaningful metric, but class-wise precision and recall (weighted) are also reported. (c) Stratified splitting is appropriate to keep this proportion in both subsets.

v. Visualization

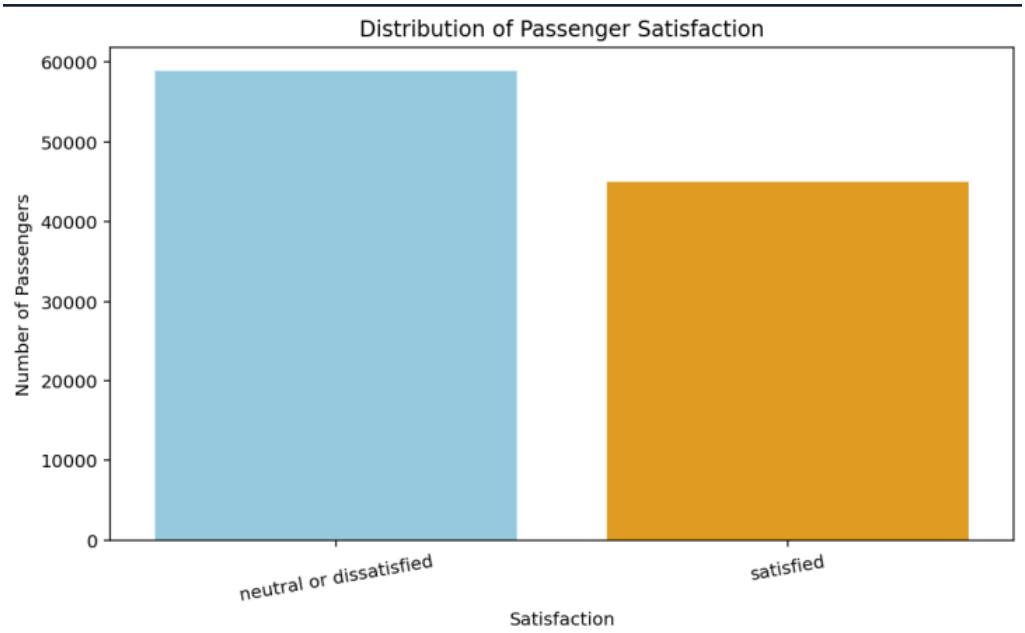


Figure 1: Distribution of Passenger Satisfaction

4.2 Passenger Age Distribution

i. Introduction

This analysis examines the age profile of the passengers in the dataset. Knowing which age ranges dominate helps interpret the results of other analyses and motivates the age-group feature created in Section 3.5.1.

ii. General Description

The variable is **Age**, a numerical variable measured in years. A histogram divides the range of a numerical variable into equal-width bins and counts the observations in each bin. A kernel density estimate (KDE) curve smooths the histogram to show the overall shape of the distribution.

iii. Specific Requirements, Functions and Formulas

The plot was created with sns.histplot(data=df, x="Age", bins=30, kde=True), which draws a 30-bin histogram with a KDE curve. The bin width is (maximum age − minimum age) / 30. Axis labels and the title were set with plt.xlabel, plt.ylabel and plt.title.

iv. Analysis Results

Figure 2 shows ages ranging from the single digits (the left-most bar begins at about 7 years) to the early eighties. The distribution is not a single smooth bell: the KDE curve shows several local peaks, with prominent bars around the mid-twenties, the late thirties to early forties and the late forties, followed by a decline after about 60 years. Very few passengers are above 70.

Key observations: (a) The passenger base is dominated by working-age adults, roughly between 20 and 60 years. (b) Children, teenagers and passengers above 70 form small groups. (c) The multi-peaked shape suggests that grouping ages into bands is more suitable than treating age as a single linear quantity. (d) Passengers under 18 are present in the data, which affects the labelling of the first age group (Section 3.5.1).

v. Visualization

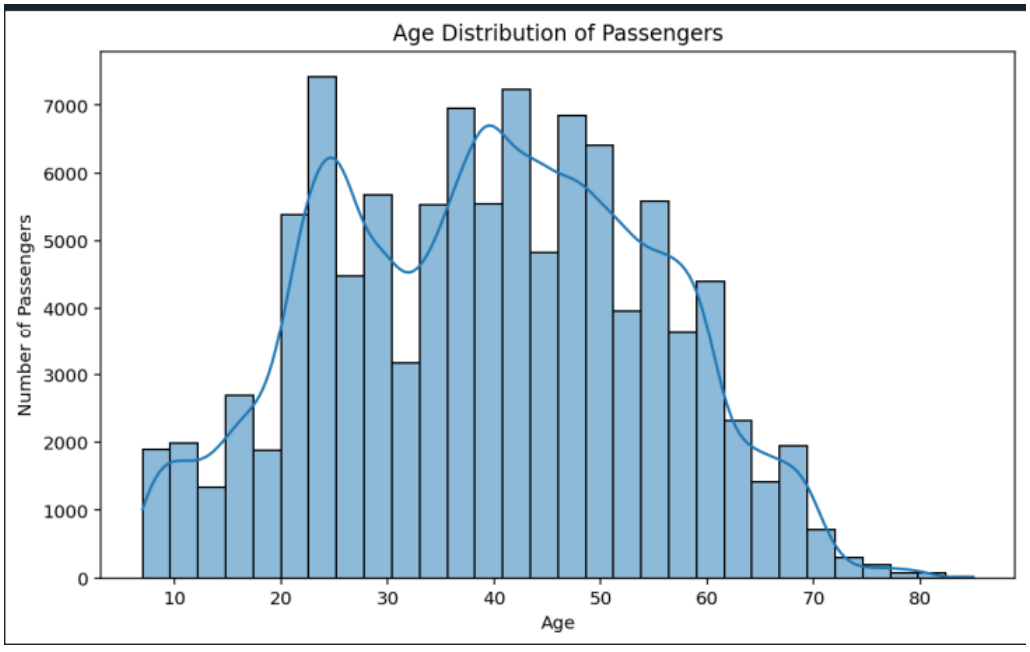


Figure 2: Age Distribution of Passengers

4.3 Satisfaction Across Travel Classes

i. Introduction

Travel class determines seat space, service level and price. This analysis compares satisfaction among the three classes to see whether the class in which a passenger travels is associated with the satisfaction outcome.

ii. General Description

The variables are **Class** (Business, Eco, Eco Plus) and **satisfaction**. A grouped count plot places a pair of bars for each class, one per satisfaction level, so the two outcomes can be compared within each class.

iii. Specific Requirements, Functions and Formulas

The plot uses sns.countplot(data=df, x="Class", hue="satisfaction"); the hue argument splits each class by satisfaction. Numerical support comes from pd.crosstab(df["Class"], df["satisfaction"]) for counts and pd.crosstab(..., normalize="index") × 100 for the percentage within each class:

$$\textit{Satisfaction \% in a class} = (\textit{satisfied passengers in class} / \textit{all passengers in class}) \times 100$$

iv. Analysis Results

In Figure 3, the Business class is the only class in which satisfied passengers outnumber neutral or dissatisfied passengers (roughly 34,500 against 15,200). In the Eco class the pattern reverses sharply, with roughly 38,000 neutral or dissatisfied passengers against roughly 8,700 satisfied passengers. Eco Plus is the smallest class and is also dominated by neutral or dissatisfied passengers (roughly 5,700 against 1,800). Percentages within each class.

Key observations: (a) Satisfaction is strongly associated with travel class. (b) Business class passengers are mostly satisfied, whereas Eco and Eco Plus passengers are mostly neutral or dissatisfied. (c) Eco is the largest group, so improving the Eco experience would affect the greatest number of passengers. (d) The Class variable is expected to be informative for classification; “Class_Eco” appears in the decision tree of Figure 10.

v. Visualization

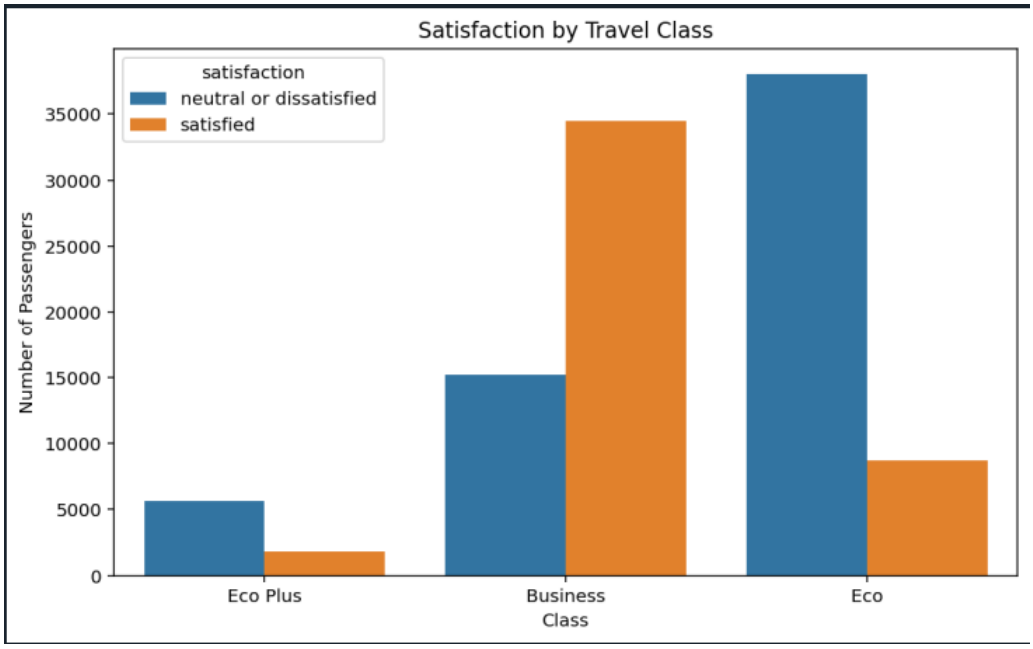


Figure 3: Satisfaction by Travel Class

4.4 Satisfaction by Type of Travel

i. Introduction

Passengers fly for different purposes, and their expectations may differ. This analysis compares satisfaction between business travellers and personal travellers.

ii. General Description

The variables are **Type of Travel** (Business travel, Personal Travel) and **satisfaction**. The display is a grouped count plot, as in Section 4.3.

iii. Specific Requirements, Functions and Formulas

The plot uses sns.countplot(data=df, x="Type of Travel", hue="satisfaction"). The numerical summary is obtained with pd.crosstab on Type of Travel and satisfaction, with the row-normalised version giving percentages within each travel type.

iv. Analysis Results

Figure 4 shows a pronounced contrast. Among personal travellers, neutral or dissatisfied passengers (roughly 29,000) far exceed satisfied passengers (roughly 3,200). Among business travellers, satisfied passengers (roughly 41,700) outnumber neutral or dissatisfied passengers (roughly 29,800). Percentages within each travel type.

Key observations: (a) Personal travellers are overwhelmingly neutral or dissatisfied. (b) Business travellers are more often satisfied, though a large number are not. (c) Type of Travel is therefore a strong predictor; the indicator "Type of Travel_Personal Travel" forms the first split of the right-hand branch of the decision tree (Figure 10). (d) Business travel and Business class (Section 4.3) are likely to overlap, so their effects are not independent.

v. Visualization

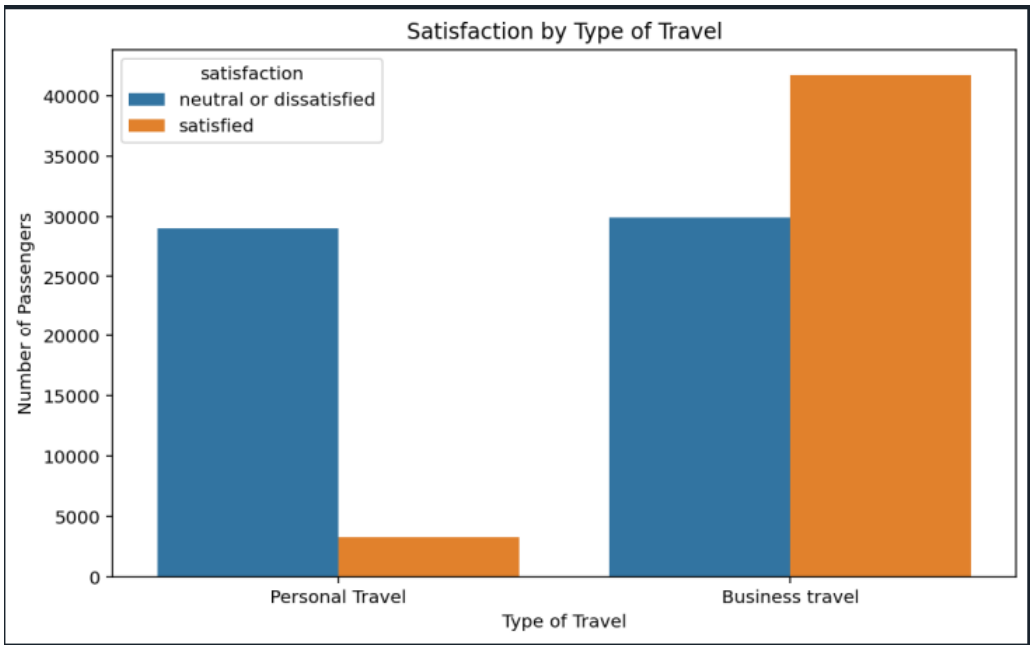


Figure 4: Satisfaction by Type of Travel

4.5 Satisfaction by Customer Type

i. Introduction

Airlines distinguish between loyal customers, who travel repeatedly with the airline, and disloyal customers. This analysis checks whether loyalty is associated with satisfaction.

ii. General Description

The variables are **Customer Type** (Loyal Customer, disloyal Customer) and **satisfaction**. The display is again a grouped count plot.

iii. Specific Requirements, Functions and Formulas

The plot uses sns.countplot(data=df, x="Customer Type", hue="satisfaction"). Because the script's objective summaries do not include a separate cross-tabulation for customer type, the percentages are not printed in the console output of the original script.

iv. Analysis Results

Figure 5 shows that loyal customers form the large majority of passengers. Among loyal customers, the two outcomes are close, with neutral or dissatisfied passengers (roughly 44,400) slightly exceeding satisfied passengers (roughly 40,500). Among disloyal customers, neutral or dissatisfied passengers (roughly 14,500) are more than three times as numerous as satisfied passengers (roughly 4,500).

Key observations: (a) Disloyal customers are much more likely to be neutral or dissatisfied. (b) Loyal customers are split almost evenly, so loyalty alone does not guarantee satisfaction. (c) Customer Type contributes to classification, as shown by the "Customer Type_disloyal Customer" split in Figure 10.

v. Visualization

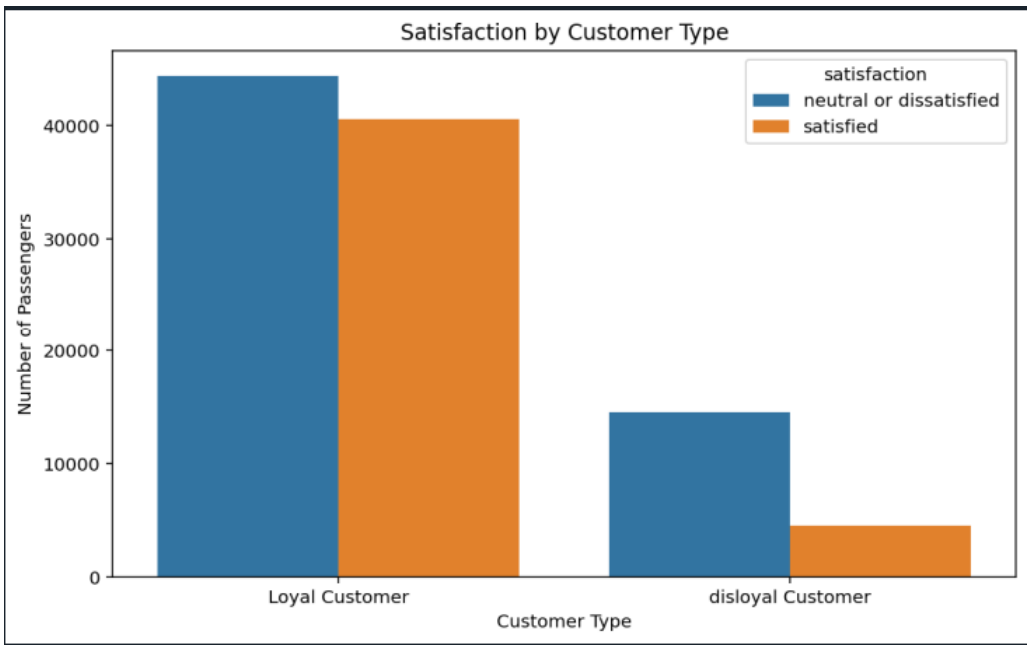


Figure 5: Satisfaction by Customer Type

4.6 Service Rating Analysis

i. Introduction

Service quality is the part of the passenger experience that an airline can most directly control. This analysis compares the average rating of fourteen service features between satisfied passengers and those who are neutral or dissatisfied, in order to identify the features that distinguish the two groups.

ii. General Description

The variables are the fourteen service-rating columns listed in Table 3 and the target **satisfaction**. Each rating is an ordinal score given by the passenger. The mean rating of each feature is computed separately for each satisfaction group, and the two group means are compared side by side.

iii. Specific Requirements, Functions and Formulas

A list named service_columns holds the fourteen column names. The group means are computed with df.groupby("satisfaction")[service_columns].mean().T, where the transpose (.T) places service features on the rows. The result is plotted with the DataFrame method plot(kind="bar", figsize=(14, 6)), and the labels are rotated with plt.xticks(rotation=50, ha="right"). The mean rating is:

$$\textit{Mean rating} = (1 / n)\ \Sigma\ \textit{rating}_i,\ \textit{computed within each satisfaction group}$$

iv. Analysis Results

In Figure 6, the satisfied group has the higher average rating for most features. The largest gaps appear for Online boarding (roughly 2.7 against 4.0), Inflight entertainment (roughly 2.9 against 4.0), Inflight wifi service (roughly 2.4 against 3.2), Seat comfort, On-board service, Leg room service, Cleanliness and Food and drink. Smaller gaps are seen for Baggage handling, Checkin service and Inflight service. Two features behave differently: Gate location has almost identical averages in both groups, and Departure/Arrival time convenient is slightly higher for the neutral or dissatisfied group. The exact averages are printed in the console under Objective 4.

Key observations: (a) Satisfied passengers rate nearly all in-flight and boarding services higher. (b) Online boarding, inflight entertainment and inflight wifi show some of the largest differences and are the most promising areas for improvement. (c) Gate

location and time convenience do not separate the groups, so they are weaker indicators of satisfaction. (d) The ordering of features in the decision tree, where Online boarding and Inflight wifi service appear near the root (Figure 10), is consistent with this analysis.

v. Visualization

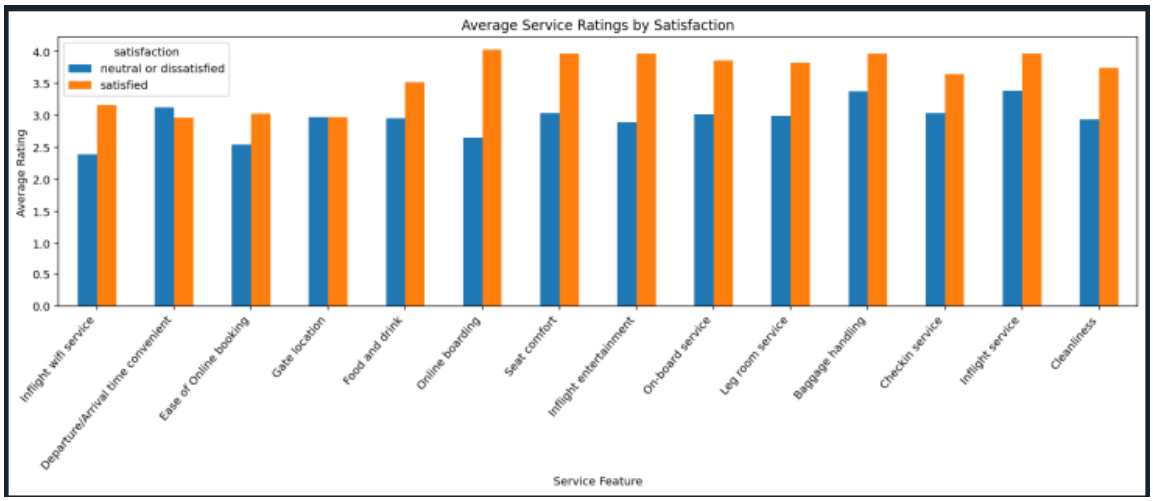


Figure 6: Average Service Ratings by Satisfaction

4.7 Correlation Analysis of Selected Features

i. Introduction

Correlation analysis quantifies how strongly pairs of numerical variables move together. This analysis examines ten selected variables: eight service ratings and the two delay measures. It reveals groups of related services and checks whether delays are related to the service ratings.

ii. General Description

The variables are Inflight wifi service, Ease of Online booking, Food and drink, Seat comfort, Inflight entertainment, On-board service, Leg room service, Cleanliness, Departure Delay in Minutes and Arrival Delay in Minutes. The Pearson coefficient r lies between −1 (perfect negative linear relationship) and +1 (perfect positive linear relationship), with values near 0 indicating no linear relationship. The target is not part of this matrix.

iii. Specific Requirements, Functions and Formulas

The matrix was computed with df[selected_columns].corr() and displayed with sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm", center=0, linewidths=0.5, annot_kws={"size": 8}). The coefficient is:

$$r = cov(X, Y) / (\sigma_X \sigma_Y)$$

iv. Analysis Results

Figure 7 shows several clusters of related features. Inflight wifi service and Ease of Online booking have a correlation of 0.72. Cleanliness is correlated with Inflight entertainment (0.69), Seat comfort (0.68) and Food and drink (0.66). Food and drink, Seat comfort and Inflight entertainment are also correlated with one another (between 0.57 and 0.62). On-board service has moderate correlations with Inflight entertainment (0.42) and Leg room service (0.36). The two delay variables are almost perfectly correlated with each other (0.96), but their correlations with every service rating lie between −0.04 and 0.01.

Key observations: (a) Comfort-related services (seat comfort, cleanliness, food, entertainment) are rated consistently by passengers, which suggests a common underlying impression of cabin quality. (b) Delays are essentially unrelated to service ratings. (c) Departure and arrival delay carry almost the same information, because a late departure usually produces a late arrival. (d) Such strongly correlated predictors can make Logistic Regression coefficients harder to interpret, whereas tree-based models are less affected.

v. Visualization

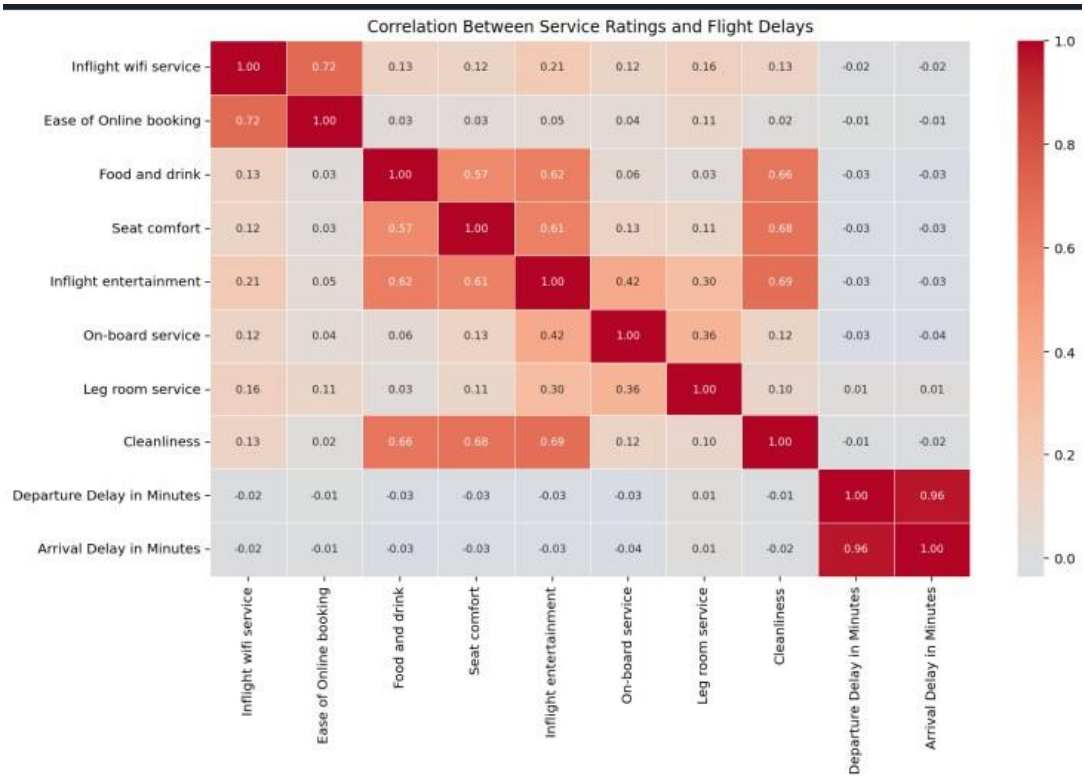


Figure 7: Correlation Between Service Ratings and Flight Delays

4.8 Arrival Delay Analysis by Passenger Satisfaction

i. Introduction

Punctuality is a common source of passenger complaints. This analysis compares the distribution of arrival delay between satisfied and neutral or dissatisfied passengers.

ii. General Description

The variables are **Arrival Delay in Minutes** (numerical) and **satisfaction**. A box plot summarises a distribution through its median, the interquartile range (the box), whiskers extending to the most extreme values within 1.5 × IQR of the quartiles, and individual points beyond the whiskers, which are outliers. Delay data is zero-inflated and strongly right-skewed: most flights have little or no delay while a few have very long delays.

iii. Specific Requirements, Functions and Formulas

The plot was drawn with sns.boxplot(data=df, x="satisfaction", y="Arrival Delay in Minutes", hue="satisfaction", palette=["#FF9999", "#66B3FF"], legend=False). The y-

axis was changed to a symmetric logarithmic scale with plt.yscale("symlog", linthresh=1). A symmetric log scale is linear for values with magnitude below the threshold (here 1) and logarithmic beyond it, so it can display zero values and very large delays together. Mean delays for each group are also printed by grouping on "satisfaction", selecting the two delay columns and calling mean().

iv. Analysis Results

In Figure 8, the lower end of both boxes is at zero minutes, and the median lies very close to zero in both groups. The upper edge of the box for neutral or dissatisfied passengers is at roughly 15 minutes, compared with roughly 8 minutes for satisfied passengers, and its upper whisker also extends further. Both groups have a dense column of outliers extending to the order of 1,000 minutes. The mean delays by group.

Key observations: (a) Most passengers in both groups experienced little or no arrival delay. (b) Neutral or dissatisfied passengers show somewhat longer delays in the upper quartile, but the distributions overlap heavily. (c) Extreme delays occur in both groups, so delay alone cannot separate the two classes. (d) This agrees with the near-zero correlation between delays and service ratings in Section 4.7. (e) The outliers were retained in the dataset (Section 3.4).

v. Visualization

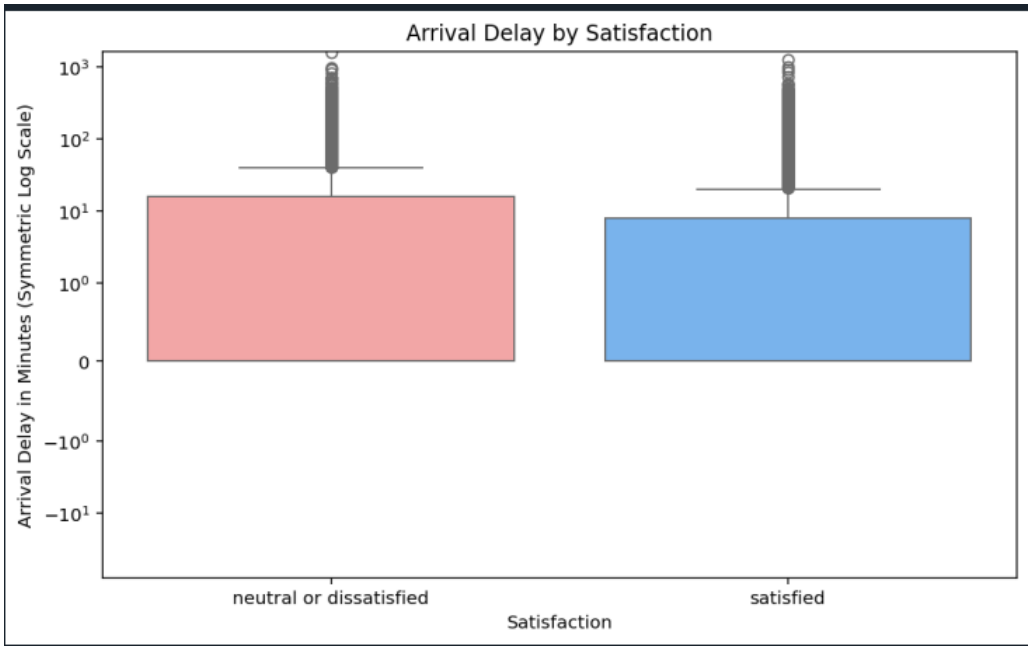


Figure 8: Arrival Delay by Satisfaction

4.9 Logistic Regression

i. Introduction

Logistic Regression is a linear model for classification. Despite its name, it predicts the probability that an observation belongs to a class. It is widely used as a baseline because it is fast, simple and interpretable (Hosmer, Lemeshow and Sturdivant, 2013). In this project it estimates the probability that a passenger is satisfied.

ii. General Description

Working principle. The model computes a weighted sum of the input features and passes it through the sigmoid function, which maps any real number to a value between 0 and 1. That value is interpreted as the probability of the positive class ("satisfied"). A passenger is predicted as satisfied when the probability is at least 0.5. The weights are learned by minimising the log-loss over the training data; scikit-learn adds L2 regularisation by default, which penalises large weights and reduces over-fitting.

Advantages: fast to train, produces probabilities, and each coefficient has a direct interpretation. **Limitations:** the decision boundary is linear, so it cannot capture non-linear interactions unless they are engineered; it is sensitive to feature scale (hence the StandardScaler) and affected by strongly correlated predictors (Section 4.7).

iii. Specific Requirements, Functions and Formulas

Mathematical explanation. For a feature vector x with weights w and bias b:

$$z = w \cdot x + b \qquad P(y = 1 \mid x) = \sigma(z) = 1 / (1 + e^{-z})$$

$$\text{Log-loss } J = -(1/m)\, \Sigma\, [\, y_i \log(p_i) + (1 - y_i) \log(1 - p_i)\,] \quad \text{(plus an L2 penalty)}$$

Important parameters. max_iter=1000 sets the maximum number of optimiser iterations, raised from the default of 100 so that the solver can converge on the scaled data. Other parameters were left at scikit-learn defaults (L2 penalty, regularisation strength C = 1.0). **Training and prediction.** The model is fitted on X_train_scaled and y_train, and predictions are made for X_test_scaled.

```
model1 = LogisticRegression(max_iter=1000)
model1.fit(X_train_scaled, y_train)
y_pred1 = model1.predict(X_test_scaled)
```

iv. Analysis Results

Figure 9 shows the confusion matrix of the Logistic Regression model on the 20,781 test passengers, and Table 9 restates the counts. Of the 11,776 passengers who were actually neutral or dissatisfied, 10,642 were classified correctly and 1,134 were wrongly predicted as satisfied (false positives). Of the 9,005 passengers who were actually satisfied, 7,591 were classified correctly and 1,414 were wrongly predicted as neutral or dissatisfied (false negatives). The model made slightly more false negatives than false positives, so it misses a somewhat larger share of truly satisfied passengers than it wrongly labels as satisfied.

Table 9: Confusion matrix of Logistic Regression

Actual \ Predicted	neutral or dissatisfied	satisfied
neutral or dissatisfied	10,642 (TN)	1,134 (FP)
satisfied	1,414 (FN)	7,591 (TP)

Table 10: Evaluation metrics of Logistic Regression on the test set

Metric	Value
Accuracy	0.877
Precision (weighted)	0.877
Recall (weighted)	0.877
F1-score (weighted)	0.877

Table 11: Classification report of Logistic Regression

Class	Precision	Recall	F1-score	Support
neutral or dissatisfied	0.88	0.90	0.89	11,776
satisfied	0.87	0.84	0.86	9,005
macro average	0.88	0.87	0.87	20,781
weighted average	0.88	0.88	0.88	20,781

v. Visualization

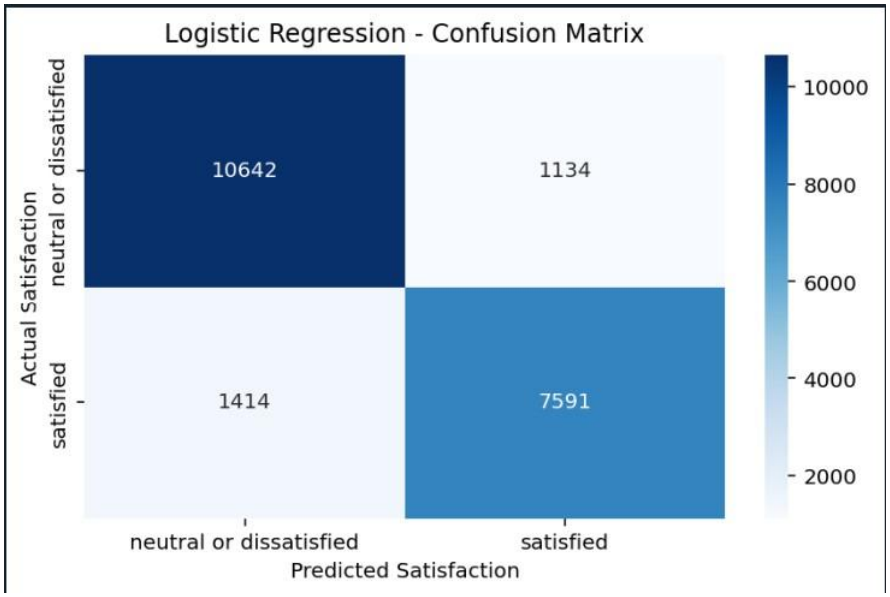


Figure 9: Logistic Regression - Confusion Matrix

4.10 Decision Tree Classifier

i. Introduction

A Decision Tree classifier predicts the class of an observation by asking a sequence of yes/no questions about its features. Each question splits the data into two groups, and the process repeats until a group is sufficiently pure or a stopping rule is reached (Breiman et al., 1984). Decision trees are easy to visualise and explain, which makes them attractive for understanding which features drive satisfaction.

ii. General Description

Working principle. The tree starts at a root node containing all training samples. At each node, the algorithm searches all features and thresholds for the split that most reduces impurity in the two child nodes. Splitting continues recursively. Each leaf stores the class that is in the majority among its training samples, and a new passenger is classified by following the questions from the root to a leaf. **Advantages:** no feature scaling is required, non-linear relationships and interactions are captured naturally, and the model is easy to interpret. **Limitations:** a deep tree can memorise the training data (over-fitting) and is sensitive to small changes in the data, which is why the depth was limited in this project.

iii. Specific Requirements, Functions and Formulas

Mathematical explanation. The default impurity measure in scikit-learn is the Gini impurity. For a node containing a fraction p_k of samples of class k:

$$Gini = 1 - \Sigma_k p_k^2$$

A pure node has Gini = 0, and a node with an even split of two classes has Gini = 0.5. The quality of a split is the weighted decrease in impurity:

$$\Delta = Gini(parent) - [\ (n_L/n)\ Gini(left) + (n_R/n)\ Gini(right)\ ]$$

Important parameters. max_depth=15 limits the number of levels in the tree and therefore the complexity of the model; random_state=42 makes tie-breaking reproducible. The default criterion (gini) is used. **Training and prediction.** The tree is fitted on the unscaled training data and used to predict the test set. For display, plot_tree was called with max_depth=3, so only the first three levels of the fitted tree are drawn.

```
model2 = DecisionTreeClassifier(max_depth=15, random_state=42)
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)
plot_tree(model2, feature_names=X.columns,
class_names=label_encoder.classes_,
          filled=True, rounded=True, max depth=3, fontsize=7)
```

iv. Analysis Results

Figure 10 shows the first three levels of the fitted tree. The root node contains all 83,123 training samples and splits on Online boarding. The branch for lower Online boarding ratings contains 41,975 samples and is split next by Inflight wifi service, while the branch for higher ratings contains 41,148 samples and is split by whether the passenger is travelling for personal reasons (Type of Travel_Personal Travel). Deeper nodes in the displayed levels use Cleanliness, Inflight entertainment, Customer Type_disloyal Customer, Class_Eco and further Online boarding and Inflight wifi thresholds. In the plot, orange nodes have a majority of neutral or dissatisfied passengers and blue nodes a majority of satisfied passengers; a darker colour means a purer node.

The features near the top of the tree, Online boarding, Inflight wifi service and Type of Travel, are consistent with the large differences seen in Sections 4.4 and 4.6. Test-set results are given in the tables below.

Table 12: Evaluation metrics of the Decision Tree on the test set

Metric	Value
Accuracy	0.95
Precision (weighted)	0.95
Recall (weighted)	0.95
F1-score (weighted)	0.95

Table 13: Classification report of the Decision Tree

Class	Precision	Recall	F1-score	Support
neutral or dissatisfied	0.95	0.97	0.96	11,776
satisfied	0.95	0.94	0.94	9,005
macro average	0.95	0.95	0.95	20,781
weighted average	0.95	0.95	0.95	20,781

Table 14: Confusion matrix of the Decision Tree

Actual \ Predicted	neutral or dissatisfied	satisfied
neutral or dissatisfied	11370	406
satisfied	583	8422

v. Visualization

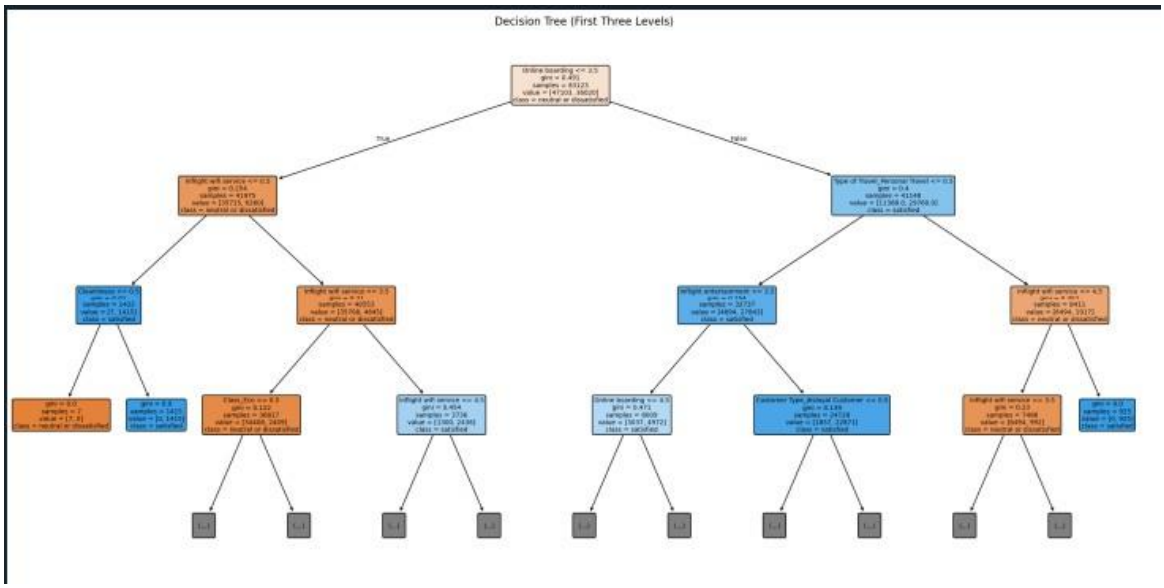


Figure 10: Decision Tree (First Three Levels)

4.11 Random Forest Classifier

i. Introduction

Random Forest is an ensemble method that combines many decision trees to obtain a more accurate and stable prediction than any single tree (Breiman, 2001). It is one of the most widely used algorithms for tabular data.

ii. General Description

Working principle. The algorithm builds many trees, each trained on a different bootstrap sample (a random sample of the training rows drawn with replacement). At each split, a tree considers only a random subset of the features, which makes the trees differ from one another. A new observation is passed through all trees and the final class is decided by majority vote. Averaging many different trees reduces the variance and the over-fitting tendency of a single deep tree. **Advantages:** high accuracy, robustness to outliers and unscaled features, and the ability to model non-linear interactions. **Limitations:** less interpretable than a single tree, slower to train and predict, and larger in memory.

iii. Specific Requirements, Functions and Formulas

Mathematical explanation. If T trees h_1, …, h_T each predict a class, the forest prediction is the majority vote:

$$\hat{y} = mode\ \{\ h_1(x),\ h_2(x),\ \ldots,\ h_T(x)\ \}$$

Each tree is trained as in Section 4.10, using the Gini impurity. **Important parameters.** n_estimators=100 sets the number of trees; random_state=42 fixes the random choices for reproducibility; n_jobs=-1 uses all available processor cores to build trees in parallel. All other parameters are scikit-learn defaults, including unrestricted tree depth. **Training and prediction.** The forest is fitted on the unscaled training data and predicts the test set.

```
model3 = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1)
model3.fit(X_train, y_train)
y_pred3 = model3.predict(X_test)
```

iv. Analysis Results

The test-set results of the Random Forest are recorded below. In the accuracy comparison (Figure 11), Random Forest has the tallest bar and is listed first, so it

achieved the highest accuracy among the four models. The script sorts the models by weighted F1-score, and Random Forest also occupies the first position in that ordering.

Table 15: Evaluation metrics of the Random Forest on the test set

Metric	Value
Accuracy	0.96
Precision (weighted)	0.96
Recall (weighted)	0.96
F1-score (weighted)	0.96

Table 16: Classification report of the Random Forest

Class	Precision	Recall	F1-score	Support
neutral or dissatisfied	0.96	0.98	0.97	11,776
satisfied	0.97	0.94	0.96	9,005
macro average	0.96	0.96	0.96	20,781
weighted average	0.96	0.96	0.96	20,781

Table 17: Confusion matrix of the Random Forest

Actual \ Predicted	neutral or dissatisfied	satisfied
neutral or dissatisfied	11488	288
satisfied	512	8493

v. Visualization

The script does not generate a separate plot for the Random Forest, because an ensemble of 100 trees cannot be drawn as a single diagram. Its performance is shown in the comparison chart of Figure 11 (Section 4.13), and the structure of an individual tree is illustrated in Figure 10.

4.12 Support Vector Machine (SVM)

i. Introduction

The Support Vector Machine is a classifier that finds the decision boundary with the largest possible margin between the two classes (Cortes and Vapnik, 1995). It is

effective in high-dimensional feature spaces and is a standard algorithm for binary classification.

ii. General Description

Working principle. In a linear SVM the boundary is a hyperplane. Among all hyperplanes that separate the classes, the SVM chooses the one that maximises the distance (margin) to the nearest training points of each class; these nearest points are called support vectors and alone determine the boundary. Because real data is rarely perfectly separable, a soft margin allows some points to violate the margin at a cost controlled by the parameter C. **Advantages:** effective in many dimensions, memory-efficient because only support vectors define the model, and strong generalisation through margin maximisation. **Limitations:** training time grows quickly with the number of samples, it needs scaled features, and it does not give probabilities directly.

iii. Specific Requirements, Functions and Formulas

Mathematical explanation. The soft-margin SVM solves:

$$\textit{minimise } \tfrac{1}{2}\|w\|^2 + C\,\Sigma\,\xi_i \quad \textit{subject to } y_i\,(w \cdot x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

$$\textit{Prediction: } \hat{y} = \textit{sign}(w \cdot x + b)$$

where ξ_i are slack variables that measure margin violations and y_i takes the values -1 and $+1$ in this formulation. The margin width is $2 / \|w\|$, so minimising $\|w\|$ maximises the margin.

Important parameters. kernel="linear" selects a linear decision boundary; the regularisation parameter C was left at its default value of 1.0. **Training and prediction.** SVM training becomes slow on large datasets, so in the script the model was trained only on the first 15,000 rows of the scaled training set, taken as min(15000, len(X_train)). Predictions were then made on the complete scaled test set of 20,781 rows. The code comment describes this subset as random, but the slice X_train_scaled[:15000] selects the first rows in order. Because the model saw only about 18% of the training data, its results are not directly comparable to those of models trained on all 83,123 rows.

```
svm_sample_size = min(15000, len(X_train))
X_svm_train = X_train_scaled[:svm_sample_size]
y_svm_train = y_train[:svm_sample_size]
model4 = SVC(kernel="linear")
model4.fit(X_svm_train, y_svm_train)
```

```
y_pred4 = model4.predict(X_test_scaled)
```

iv. Analysis Results

The test-set results of the SVM are recorded below. In Figure 11 the SVM bar is similar in height to that of Logistic Regression and clearly below those of the two tree-based models, which is plausible because both Logistic Regression and the linear SVM use a linear boundary.

Table 18: Evaluation metrics of the Support Vector Machine on the test set

Metric	Value
Accuracy	0.87
Precision (weighted)	0.87
Recall (weighted)	0.87
F1-score (weighted)	0.87

Table 19: Classification report of the Support Vector Machine

Class	Precision	Recall	F1-score	Support
neutral or dissatisfied	0.88	0.91	0.89	11,776
satisfied	0.87	0.84	0.86	9,005
macro average	0.88	0.87	0.88	20,781
weighted average	0.88	0.88	0.88	20,781

Table 20: Confusion matrix of the Support Vector Machine

Actual \ Predicted	neutral or dissatisfied	satisfied
neutral or dissatisfied	10695	1081
satisfied	1449	7556

v. Visualization

The script does not produce a separate plot for the SVM; its accuracy is shown in Figure 11.

4.13 Model Evaluation and Comparison

i. Introduction

A classifier must be assessed on data it has not seen during training. This analysis defines the evaluation metrics used in the project, compares the four models on the test set and identifies the best-performing model.

ii. General Description

Confusion matrix. A confusion matrix tabulates predicted against actual classes. With "satisfied" as the positive class: TP (true positives) are satisfied passengers predicted as satisfied; TN (true negatives) are neutral or dissatisfied passengers predicted as such; FP (false positives) are neutral or dissatisfied passengers wrongly predicted as satisfied; and FN (false negatives) are satisfied passengers wrongly predicted as neutral or dissatisfied.

Weighted averaging. The script computes precision, recall and F1-score with average="weighted": the metric is calculated for each class and then averaged using the number of true samples (support) of each class as weights. This accounts for the class imbalance. A consequence of this definition is that weighted recall is numerically equal to accuracy.

iii. Specific Requirements, Functions and Formulas

The metrics used are defined as follows.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP) \qquad Recall = TP / (TP + FN)$$

$$F1\text{-}score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

$$Weighted\ metric = \Sigma_k\ (support_k / N) \times metric_k$$

Accuracy is the fraction of all predictions that are correct; it is easy to understand but can mislead when classes are imbalanced. **Precision** answers "of the passengers predicted as satisfied, how many truly are?", and matters when false alarms are costly. **Recall** answers "of the truly satisfied passengers, how many did the model find?", and matters when missing positives is costly. **F1-score** is the harmonic mean of precision and recall and gives a single balanced measure. The **classification report** (classification_report) lists precision, recall, F1-score and support for each class

together with macro and weighted averages. In the script, the models are compared on accuracy and weighted F1-score, and the best model is the one with the highest weighted F1-score.

```
accuracy_score(y_test, y_pred)
precision_score(y_test, y_pred, average="weighted")
recall_score(y_test, y_pred, average="weighted")
f1_score(y_test, y_pred, average="weighted")
confusion_matrix(y_test, y_pred)
classification_report(y_test, y_pred,
target_names=label_encoder.classes_)
```

iv. Analysis Results

Table 21 is the model comparison table, to be completed with the values printed by the script ("MODEL COMPARISON"). The models are listed in the order shown by the bar chart in Figure 11, which the script sorts by weighted F1-score in descending order.

Table 21: Comparison of the four classification models on the test set

Rank	Model	Accuracy	Precision (weighted)	Recall (weighted)	F1-score (weighted)
1	Random Forest	0.96	0.96	0.96	0.96
2	Decision Tree	0.95	0.95	0.95	0.95
3	Support Vector Machine	0.87	0.87	0.87	0.87
4	Logistic Regression	0.87	0.87	0.87	0.87

Figure 11 shows that Random Forest and Decision Tree reach accuracies of roughly 0.95 to 0.96, whereas Support Vector Machine and Logistic Regression are both at roughly 0.88 (values read from the bar heights; exact scores belong in Table 21). The two tree-based models therefore outperform the two linear models by a clear margin. This indicates that the relationship between passenger features and satisfaction contains non-linear patterns and interactions, such as the combined effect of travel type, class and service ratings, that linear boundaries cannot fully represent.

Best-performing model. According to the order of the bars in Figure 11, which the script derives from the weighted F1-score, and the height of the bars, **Random Forest** is the best-performing model of this project, followed by the Decision Tree. Its exact accuracy is **0.9615.** This conclusion holds for the single train–test split used here; the SVM was also trained on a reduced sample (Section 4.12), and no cross-validation was performed.

v. Visualization

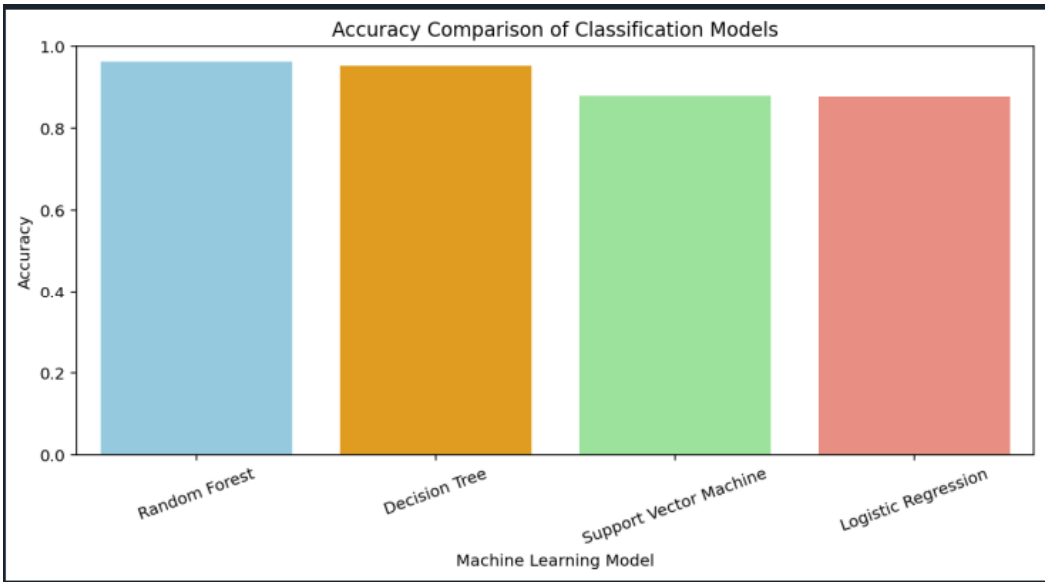


Figure 11: Accuracy Comparison of Classification Models

5. Conclusion

This project analysed an airline passenger satisfaction dataset obtained from Kaggle and built four classification models to predict whether a passenger is satisfied or neutral or dissatisfied. The work covered data cleaning, exploratory analysis, feature engineering, model training and evaluation. This chapter summarises the findings.

5.1 Data Preparation

The raw data was prepared by removing the index and identifier columns, stripping stray spaces from text values, removing duplicate records, and filling missing values with the mean for numerical columns and the mode for categorical columns. After preparation, 103,904 records were available for modelling, which were divided into 83,123 training and 20,781 test records using a stratified 80:20 split.

5.2 Exploratory Findings and Satisfaction Patterns

- **Overall distribution.** Neutral or dissatisfied passengers form the larger class (roughly 59,000 against roughly 45,000), so the classes are moderately imbalanced (Figure 1).
- **Age.** Passengers are mainly working-age adults between roughly 20 and 60 years, with few passengers above 70 (Figure 2).
- **Travel class.** Satisfaction is closely tied to class: Business class passengers are mostly satisfied, whereas the majority of Eco and Eco Plus passengers are neutral or dissatisfied (Figure 3).
- **Type of travel.** Personal travellers are overwhelmingly neutral or dissatisfied, while business travellers are more often satisfied (Figure 4).
- **Customer type.** Disloyal customers are far more likely to be neutral or dissatisfied; loyal customers are split almost evenly (Figure 5).
- **Service ratings.** Satisfied passengers rate almost all services higher. The largest differences are for Online boarding, Inflight entertainment and Inflight wifi service, while Gate location shows almost no difference (Figure 6).
- **Correlation.** Comfort-related ratings are correlated with each other, Inflight wifi service and Ease of Online booking are correlated (0.72), and departure and

arrival delay are almost perfectly correlated (0.96) but unrelated to service ratings (Figure 7).

- **Arrival delay.** Most passengers experienced little or no delay. Neutral or dissatisfied passengers show somewhat longer delays in the upper quartile, but the distributions overlap heavily (Figure 8).

Taken together, the analyses show that satisfaction is shaped by who is travelling (travel purpose, class, loyalty) and by how they rate key services, and much less by delay alone.

5.3 Machine Learning Implementation and Model Comparison

Four algorithms were implemented with scikit-learn: Logistic Regression, Decision Tree (maximum depth 15), Random Forest (100 trees) and a linear Support Vector Machine trained on 15,000 training rows. Logistic Regression and SVM used standardised features, whereas the tree-based models used unscaled features. The models were evaluated with accuracy, weighted precision, weighted recall, weighted F1-score, confusion matrices and classification reports. The Logistic Regression model correctly classified 10,642 neutral or dissatisfied and 7,591 satisfied passengers out of 20,781 test passengers. The accuracy comparison shows that the two tree-based models, Random Forest and Decision Tree, perform clearly better than the two linear models, and that **Random Forest is the best-performing model** of the project. The nodes near the top of the decision tree (Online boarding, Inflight wifi service and Type of Travel) agree with the main differences found in the exploratory analysis. The final numerical scores of all models are given in Table 21 **.**

5.4 Key Learning Outcomes

- Practical experience in cleaning a real-world survey dataset and justifying each preprocessing decision.
- Use of pandas, Matplotlib and seaborn to explore data and communicate results through visualizations.
- Understanding of feature engineering, including one-hot encoding with dummy-variable avoidance, label encoding and standardisation fitted only on training data.

- Hands-on implementation and comparison of linear, tree-based, ensemble and margin-based classifiers, and understanding of how to evaluate them with several metrics.
- Awareness of limitations, such as the single train–test split, the reduced training sample for SVM, and the difference between association and causation.

6. Future Scope

The improvements listed in this chapter are **proposed work**. They have **not** been implemented in this project. Table 22 separates what was completed from what is proposed, and the sections that follow describe the proposals.

Table 22: Completed work and proposed improvements

Area	Completed in this project	Proposed (future work)
Preprocessing	Column removal, duplicate removal, mean/mode imputation, text stripping	Outlier treatment, advanced imputation
Features	Age group, one-hot encoding, label encoding, scaling	Systematic feature selection, interaction features
Models	Four classifiers with mostly default parameters	Hyperparameter tuning, cross-validation, boosting
Delivery	Offline Python script and static figures	Interactive dashboard, web application, real-time prediction

6.1 Hyperparameter Tuning and Cross-Validation

The models were trained with default or manually chosen parameters on a single train–test split. Techniques such as grid search or randomised search with k-fold cross-validation (GridSearchCV, RandomizedSearchCV) could be used to tune parameters such as tree depth, number of trees, the regularisation strength C and the SVM kernel, and to obtain more reliable performance estimates.

6.2 Feature Selection and Model Optimisation

Feature selection methods, including feature importance from Random Forest, recursive feature elimination or correlation-based filtering, could remove redundant features such as one of the two nearly identical delay variables, and may simplify the models. The SVM could be trained on the complete training set or with non-linear kernels, and ensemble methods such as gradient boosting could be compared with Random Forest. Class-imbalance handling and outlier treatment for the delay columns could also be explored.

6.3 Interactive Dashboard

The static figures of this report could be turned into an interactive dashboard (for example with Plotly Dash or Streamlit) that lets airline analysts filter by class, travel type or age group and inspect service ratings dynamically.

6.4 Web Application and Real-Time Prediction

The best-performing model could be saved and deployed behind a web application so that a user can enter passenger and service details and receive a predicted satisfaction outcome. In a further step, the model could be connected to a live feedback stream to provide real-time satisfaction prediction, allowing airlines to respond to dissatisfied passengers during or immediately after a flight. Such a deployment would require validation on new data and monitoring for changes in passenger behaviour.

GitHub Link : https://tinyurl.com/dkpzj4fb

LinkedIn Link : https://tinyurl.com/5fcs2ckc

7. References

[1] *Airline Passenger Satisfaction*. Kaggle. https://tinyurl.com/2r39xcwp Accessed .

[2] Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/

[3] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, pp. 56–61.

[4] The pandas development team. *pandas Documentation*. https://pandas.pydata.org/docs/

[5] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.

[6] Matplotlib development team. *Matplotlib Documentation*. https://matplotlib.org/stable/

[7] Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.

[8] Waskom, M. et al. *seaborn Documentation*. https://seaborn.pydata.org/

[9] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[10] scikit-learn developers. *scikit-learn User Guide and API Reference*. https://scikit-learn.org/stable/

[11] Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

[12] Hosmer, D. W., Lemeshow, S. and Sturdivant, R. X. (2013). *Applied Logistic Regression*, 3rd ed. Wiley.

[13] Breiman, L., Friedman, J. H., Olshen, R. A. and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.

[14] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[15] Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

Appendix A: Python Source Code

The complete script used in the project is reproduced below. Only the local file path in the data-loading statement has been replaced by a generic placeholder; no other change has been made.

```
# ============================================================
# AIRLINE PASSENGER SATISFACTION PROJECT
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    precision_score, recall_score, f1_score
)

# ------------------------------------------------------------------------------
# STEP 1: LOAD THE DATASET
# ------------------------------------------------------------------------------

df = pd.read_csv("<path-to-dataset>/Airline Passenger Satisfaction.csv")

print("First five rows:")
print(df.head())

print("\nDataset information:")
df.info()

print("\nDescriptive statistics:")
print(df.describe(include="all"))

# ------------------------------------------------------------------------------
# STEP 2: DROP UNNAMED COLUMN AND ID
# ------------------------------------------------------------------------------

df.drop(columns=["Unnamed: 0", "id"], inplace=True, errors="ignore")
print("\nShape after dropping Unnamed column and id:", df.shape)

# ------------------------------------------------------------------------------
# STEP 3: CLEAN TEXT AND REMOVE DUPLICATES
# ------------------------------------------------------------------------------

for column in df.select_dtypes(include="object").columns:
    df[column] = df[column].str.strip()

print("\nDuplicate rows:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Shape after removing duplicates:", df.shape)

# ------------------------------------------------------------------------------
# STEP 4: HANDLE MISSING VALUES
# Numerical columns: mean
```

```
# Categorical columns: mode
# ------------------------------------------------------------------------------


print("\nMissing values before handling:")
print(df.isnull().sum())

numerical_columns = df.select_dtypes(include="number").columns
categorical_columns = df.select_dtypes(include="object").columns

for column in numerical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mean())

for  column  in  categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mode()[0])

print("\nMissing values after handling:")
print(df.isnull().sum())

# ------------------------------------------------------------------------------
# STEP 5: CORRELATION, COVARIANCE AND OUTLIERS
# ------------------------------------------------------------------------------

print("\nCorrelation matrix:")
print(df.corr(numeric_only=True).round(2))

print("\nCovariance matrix:")
print(df.cov(numeric_only=True).round(2))

print("\nSkewness:")
print(df.select_dtypes(include="number").skew().round(2))

print("\nKurtosis:")
print(df.select_dtypes(include="number").kurt().round(2))

for column in ["Age", "Flight Distance",
               "Departure Delay in Minutes", "Arrival Delay in Minutes"]:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    outliers = df[
        (df[column] < Q1 - 1.5 * IQR) |
        (df[column] > Q3 + 1.5 * IQR)
    ]
    print(f"Outliers in {column}: {len(outliers)}")

# ------------------------------------------------------------------------------
# STEP 6: EXPLORATORY DATA ANALYSIS AND VISUALIZATION
# ------------------------------------------------------------------------------

# Plot 1: Overall passenger satisfaction

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="satisfaction", palette=["skyblue", "orange"])
plt.title("Distribution of Passenger Satisfaction")
plt.xlabel("Satisfaction")
plt.ylabel("Number of Passengers")
plt.xticks(rotation=10)
plt.tight_layout()
plt.show()

# Plot 2: Age distribution
plt.figure(figsize=(8, 5))
sns.histplot(data=df, x="Age", bins=30, kde=True)
```

```
plt.title("Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()

# Plot 3: Satisfaction by class
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="Class", hue="satisfaction")
plt.title("Satisfaction by Travel Class")
plt.xlabel("Class")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()

# Plot 4: Satisfaction by type of travel
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="Type of Travel", hue="satisfaction")
plt.title("Satisfaction by Type of Travel")
plt.xlabel("Type of Travel")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()

# Plot 5: Satisfaction by customer type
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="Customer Type", hue="satisfaction")
plt.title("Satisfaction by Customer Type")
plt.xlabel("Customer Type")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()

# Plot 6: Service ratings by satisfaction
service_columns = [
    "Inflight wifi service",
    "Departure/Arrival time convenient",
    "Ease of Online booking",
    "Gate location",
    "Food and drink",
    "Online boarding",
    "Seat comfort",
    "Inflight entertainment",
    "On-board service",
    "Leg room service",
    "Baggage handling",
    "Checkin service",
    "Inflight service",
    "Cleanliness"
]

service_means = df.groupby("satisfaction")[service_columns].mean().T
service_means.plot(kind="bar", figsize=(14, 6))
plt.title("Average Service Ratings by Satisfaction")
plt.xlabel("Service Feature")
plt.ylabel("Average Rating")
plt.xticks(rotation=50, ha="right")
plt.tight_layout()
plt.show()

# Correlation Heatmap
# Correlation heatmap for selected features
selected_columns = [
    "Inflight wifi service",
    "Ease of Online booking",
    "Food and drink",
    "Seat comfort",
    "Inflight entertainment",
    "On-board service",
```

```
    "Leg room service",
    "Cleanliness",
    "Departure Delay in Minutes",
    "Arrival Delay in Minutes"
]

correlation_matrix = df[selected_columns].corr()

plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    linewidths=0.5,
    annot_kws={"size": 8}
)

plt.title("Correlation Between Service Ratings and Flight Delays")
plt.tight_layout()
plt.show()

# Plot 8: Arrival delay by satisfaction
# Box Plot: Arrival Delay by Satisfaction
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="satisfaction",
    y="Arrival Delay in Minutes",
    hue="satisfaction",
    palette=["#FF9999", "#66B3FF"],
    legend=False
)

plt.yscale("symlog", linthresh=1)
plt.title("Arrival Delay by Satisfaction")
plt.xlabel("Satisfaction")
plt.ylabel("Arrival Delay in Minutes (Symmetric Log Scale)")
plt.tight_layout()
plt.show()

# ------------------------------------------------------------------------------
# STEP 7: FIVE PROJECT OBJECTIVES
# ------------------------------------------------------------------------------

# Objective 1: Overall satisfaction
print("\nOBJECTIVE 1: Overall Passenger Satisfaction")
print(df["satisfaction"].value_counts())
print("\nSatisfaction percentage:")
print((df["satisfaction"].value_counts(normalize=True) * 100).round(2))

# Objective 2: Satisfaction by class
print("\nOBJECTIVE 2: Satisfaction by Class")
print(pd.crosstab(df["Class"], df["satisfaction"]))
print("\nPercentage within each class:")
print((pd.crosstab(df["Class"], df["satisfaction"], normalize="index") *
100).round(2))

# Objective 3: Satisfaction by type of travel
print("\nOBJECTIVE 3: Satisfaction by Type of Travel")
print(pd.crosstab(df["Type of Travel"], df["satisfaction"]))
print("\nPercentage within each travel type:")
print((pd.crosstab(df["Type of Travel"], df["satisfaction"], normalize="index") *
100).round(2))
```

```
# Objective 4: Service ratings by satisfaction
print("\nOBJECTIVE 4: Average Service Ratings by Satisfaction")
print(df.groupby("satisfaction")[service_columns].mean().T.round(2))

# Objective 5: Flight delays by satisfaction
print("\nOBJECTIVE 5: Flight Delays by Satisfaction")
print(df.groupby("satisfaction")[
    ["Departure Delay in Minutes", "Arrival Delay in Minutes"]
].mean().round(2))

# ------------------------------------------------------------------------------
# STEP 8: FEATURE ENGINEERING
# ------------------------------------------------------------------------------

df["Age Group"] = pd.cut(
    df["Age"],
    bins=[0, 24, 39, 59, 100],
    labels=["18-24", "25-39", "40-59", "60+"]
)

# ------------------------------------------------------------------------------
# STEP 9: PREPARE DATA FOR CLASSIFICATION
# ------------------------------------------------------------------------------

# satisfaction is the target we want to predict.
X = df.drop("satisfaction", axis=1)
y = df["satisfaction"]

# Convert text columns into 0/1 columns using pandas.
# This is a simple alternative to OneHotEncoder.
X = pd.get_dummies(X, drop_first=True)

# Convert the target labels into numbers.
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

print("\nTarget label mapping:")
print(dict(zip(label_encoder.classes_,
               label_encoder.transform(label_encoder.classes_))))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# StandardScaler is used for Logistic Regression and SVM.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ------------------------------------------------------------------------------
# STEP 10: MODEL 1 - LOGISTIC REGRESSION
# ------------------------------------------------------------------------------

model1 = LogisticRegression(max_iter=1000)
model1.fit(X_train_scaled, y_train)
y_pred1 = model1.predict(X_test_scaled)

print("\nLOGISTIC REGRESSION")
print("Accuracy:", accuracy_score(y_test, y_pred1))
print("Precision:", precision_score(y_test, y_pred1, average="weighted"))
print("Recall:", recall_score(y_test, y_pred1, average="weighted"))
print("F1 Score:", f1_score(y_test, y_pred1, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred1))
print("Classification Report:\n",
      classification_report(y_test, y_pred1,
```

```
                              target_names=label_encoder.classes_))

# Logistic Regression - Confusion Matrix Plot
plt.figure(figsize=(6, 4))
sns.heatmap(
    confusion_matrix(y_test, y_pred1),
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)
plt.title("Logistic Regression - Confusion Matrix")
plt.xlabel("Predicted Satisfaction")
plt.ylabel("Actual Satisfaction")
plt.tight_layout()
plt.show()

# ------------------------------------------------------------------------------
# STEP 11: MODEL 2 - DECISION TREE
# ------------------------------------------------------------------------------

model2 = DecisionTreeClassifier(max_depth=15, random_state=42)
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)

print("\nDECISION TREE")
print("Accuracy:", accuracy_score(y_test, y_pred2))
print("Precision:", precision_score(y_test, y_pred2, average="weighted"))
print("Recall:", recall_score(y_test, y_pred2, average="weighted"))
print("F1 Score:", f1_score(y_test, y_pred2, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred2))
print("Classification Report:\n",
      classification_report(y_test, y_pred2,
                            target_names=label_encoder.classes_))

plt.figure(figsize=(18, 9))
plot_tree(
    model2,
    feature_names=X.columns,
    class_names=label_encoder.classes_,
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=7
)
plt.title("Decision Tree (First Three Levels)")
plt.tight_layout()
plt.show()

# ------------------------------------------------------------------------------
# STEP 12: MODEL 3 - RANDOM FOREST
# ------------------------------------------------------------------------------

model3 = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
model3.fit(X_train, y_train)
y_pred3 = model3.predict(X_test)

print("\nRANDOM FOREST")
print("Accuracy:", accuracy_score(y_test, y_pred3))
print("Precision:", precision_score(y_test, y_pred3, average="weighted"))
print("Recall:", recall_score(y_test, y_pred3, average="weighted"))
print("F1 Score:", f1_score(y_test, y_pred3, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred3))
print("Classification Report:\n",
```

```
      classification_report(y_test, y_pred3,
                            target_names=label_encoder.classes_))

# ------------------------------------------------------------------------------
# STEP 13: MODEL 4 - SUPPORT VECTOR MACHINE (SVM)
# ------------------------------------------------------------------------------

# SVM can take a long time on a very large dataset.
# Train it on a random sample of the training rows to keep runtime practical.
svm_sample_size = min(15000, len(X_train))
X_svm_train = X_train_scaled[:svm_sample_size]
y_svm_train = y_train[:svm_sample_size]

model4 = SVC(kernel="linear")
model4.fit(X_svm_train, y_svm_train)
y_pred4 = model4.predict(X_test_scaled)

print("\nSUPPORT VECTOR MACHINE")
print("SVM training sample size:", svm_sample_size)
print("Accuracy:", accuracy_score(y_test, y_pred4))
print("Precision:", precision_score(y_test, y_pred4, average="weighted"))
print("Recall:", recall_score(y_test, y_pred4, average="weighted"))
print("F1 Score:", f1_score(y_test, y_pred4, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred4))
print("Classification Report:\n",
      classification_report(y_test, y_pred4,
                            target_names=label_encoder.classes_))

# ------------------------------------------------------------------------------
# STEP 14: COMPARE MODELS AND SELECT THE BEST MODEL
# ------------------------------------------------------------------------------

models = [
    "Logistic Regression",
    "Decision Tree",
    "Random Forest",
    "Support Vector Machine"
]

accuracies = [
    accuracy_score(y_test, y_pred1),
    accuracy_score(y_test, y_pred2),
    accuracy_score(y_test, y_pred3),
    accuracy_score(y_test, y_pred4)
]

f1_scores = [
    f1_score(y_test, y_pred1, average="weighted"),
    f1_score(y_test, y_pred2, average="weighted"),
    f1_score(y_test, y_pred3, average="weighted"),
    f1_score(y_test, y_pred4, average="weighted")
]

model_comparison = pd.DataFrame({
    "Model": models,
    "Accuracy": accuracies,
    "Weighted F1 Score": f1_scores
})

model_comparison = model_comparison.sort_values(
    by="Weighted F1 Score", ascending=False
)

print("\nMODEL COMPARISON")
print(model_comparison.to_string(index=False))

best_model_name = model_comparison.iloc[0]["Model"]
```

```
best_accuracy = model_comparison.iloc[0]["Accuracy"]

print("\nBEST-FIT MODEL:", best_model_name)
print("Best model accuracy:", round(best_accuracy, 4))

plt.figure(figsize=(9, 5))

sns.barplot(
    data=model_comparison,
    x="Model",
    y="Accuracy",
    hue="Model",
    palette=["skyblue", "orange", "lightgreen", "salmon"],
    legend=False
)

plt.title("Accuracy Comparison of Classification Models")
plt.xlabel("Machine Learning Model")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()

# ------------------------------------------------------------------------------
# STEP 15: FINAL CONCLUSIONS AND RECOMMENDATIONS
# ------------------------------------------------------------------------------

print("\nFINAL CONCLUSIONS AND RECOMMENDATIONS")

print("\n1. Overall satisfaction percentage:")
print((df["satisfaction"].value_counts(normalize=True) * 100).round(2))

print("\n2. Satisfaction percentage by class:")
print((pd.crosstab(df["Class"], df["satisfaction"], normalize="index") *
100).round(2))

print("\n3. Satisfaction percentage by travel type:")
print((pd.crosstab(df["Type of Travel"], df["satisfaction"], normalize="index") *
100).round(2))

print("\n4. Average service ratings by satisfaction:")
print(df.groupby("satisfaction")[service_columns].mean().T.round(2))

print("\n5. Average delays by satisfaction:")
print(df.groupby("satisfaction")[
    ["Departure Delay in Minutes", "Arrival Delay in Minutes"]
].mean().round(2))

print("\n6. Best model:", best_model_name)
print("   Accuracy:", round(best_accuracy, 4))

print("\nRecommendations:")
print("- Improve service areas where dissatisfied passengers give lower
ratings.")
print("- Compare satisfaction between travel classes and types of travel.")
print("- Monitor departure and arrival delays.")
print("- Validate the selected model on new data before using it in practice.")
```